

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS
FACULTAD DE INGENIERÍA TAMPICO

Proyecto - Marco Teórico sobre el Problema de la Canasta Básica
ADMINISTRACIÓN DE BASES DE DATOS CORPORATIVAS

ALUMNOS:

MACIAS TIJERINA GUILLERMO ANTONIO
POLANCO TIJERINA JOSE LUIS
PECERO ACEVEDO HECTOR HUMBERTO
GONZALES GONZALES OSWALDO
PONCE MEDINA JESUS
ORETEGA RESENDIZ LUIS
FERNANDO RODRIGUEZ HERNANDEZ
JUAN JULIAN

SEMESTRE: 7° GRUPO: "M"

CARRERA:

ING. EN SISTEMAS COMPUTACIONALES

PROFESOR:

Hernández Marín Juan Carlos

Entrega de Trabajo
MAYO 2026

Base de Datos

Las bases de datos constituyen sistemas fundamentales que consolidan una colección de datos interrelacionados, diseñados para almacenar información de manera eficiente y estructurada. Su objetivo primordial es preservar esta información, asegurando su integridad, consistencia y disponibilidad para múltiples usuarios y aplicaciones. En el contexto de la ingeniería de software, una base de datos se define como un repositorio organizado de datos que permite realizar operaciones de creación, lectura, actualización y eliminación (conocidas como operaciones CRUD) de forma controlada y segura.

El sistema de gestión de bases de datos (SGBD) es el software encargado de administrar este repositorio. Un SGBD comprende un conjunto de instrucciones de software interrelacionadas, donde cada una se atribuye la responsabilidad de un cometido específico: desde la definición de la estructura de los datos hasta el control de acceso concurrente y la recuperación ante fallos. El propósito elemental de un SGBD es proporcionar una forma fiable de almacenar y recuperar la información contenida en la base de datos, aislando al usuario de los detalles físicos del almacenamiento.

En nuestro proyecto específicamente, utilizamos PostgreSQL como SGBD, ejecutándose dentro de un contenedor Docker. PostgreSQL es un sistema de base de datos relacional de código abierto reconocido por su robustez, cumplimiento de estándares SQL y capacidad para manejar grandes volúmenes de datos. La elección de PostgreSQL se justifica por su soporte completo de claves foráneas, transacciones ACID, y su excelente rendimiento en operaciones de carga masiva que son esenciales para nuestro proyecto de 400,000 transacciones y aproximadamente 6.8 millones de líneas de detalle.

***ANEXO:** Las bases de datos no solo almacenan información, sino que representan la base estructural de los sistemas de información modernos. Su importancia trasciende el almacenamiento para convertirse en el motor de la toma de decisiones basada en datos.*

Impacto que Tiene en una Organización

Las bases de datos representan un pilar fundamental en las organizaciones modernas, actuando como el núcleo para la gestión, almacenamiento y análisis de información crítica. En el sector retail específicamente, cada transacción de venta, cada movimiento de inventario y cada interacción con el cliente genera datos que, correctamente almacenados y analizados, se convierten en ventaja competitiva.

Las bases de datos permiten ordenar, proteger y dar sentido a la información que sustenta la actividad diaria y la toma de decisiones. Proporcionan una estructura organizada para manejar grandes volúmenes de datos, facilitan la generación de reportes, y habilitan el análisis de patrones que sería imposible realizar manualmente.

Principales Impactos y Beneficios

Toma de decisiones basada en datos: Las bases de datos ofrecen datos precisos, actualizados y accesibles, reduciendo las decisiones basadas en intuición y permitiendo análisis informados. Mediante herramientas de business intelligence, una organización puede pasar de «creo que estos productos se venden juntos» a «el 85% de las ventas que incluyen Bakery también incluyen Fruits & Vegetables, con un soporte de 0.85 y lift de 1.02». En nuestro proyecto, esto se traduce en poder analizar 400,000 ventas para identificar patrones reales de compra.

Aumento de eficiencia y productividad: Automatizan procesos repetitivos, centralizan la información y evitan duplicidades, liberando tiempo para el personal. La automatización de pipelines de datos mediante herramientas como SQLAlchemy permite que la carga masiva de 6.8 millones de registros se complete en minutos. En nuestro proyecto, la carga usa `chunksize=10000` y `method='multi'` para optimizar la inserción.

Ventaja competitiva: Facilitan la personalización, como marketing dirigido basado en segmentación de clientes, detección de oportunidades de negocio y anticipación a tendencias. En e-commerce, el análisis de reglas de asociación permite crear sistemas de recomendación tipo «También te puede gustar». Nuestro Ejercicio 3 (KNN Hamming) implementa exactamente este caso de uso.

Mejor conocimiento del cliente: Permiten segmentar clientes, analizar comportamientos y mejorar la experiencia del usuario. Mediante análisis de cohortes o modelos de clasificación como KNN, las empresas pueden identificar clientes de alto valor y diseñar estrategias diferenciadas. Nuestro Ejercicio 2 clasifica clientes en «Gasto Alto» y «Gasto Bajo» usando distancia Euclidiana.

Reducción de costos y riesgos: Mejoran la seguridad de los datos, cumplen normativas de privacidad y minimizan errores por manipulación manual. Las restricciones de integridad referencial (FOREIGN KEY) en PostgreSQL garantizan que cada detalle de venta apunte a un artículo y una venta válidos, evitando datos huérfanos o inconsistentes.

Soporte a la transformación digital: Son la base para big data, IA, analítica avanzada y estrategias data-driven. En 2026, tendencias como lakehouse (combinando data lakes y warehouses) y data mesh (propiedad descentralizada de datos) están transformando cómo las organizaciones gestionan sus datos. Nuestro proyecto demuestra cómo una base de datos relacional alimenta directamente algoritmos de Machine Learning.

ANEXO: *Estos beneficios deben analizarse de forma conjunta, ya que la automatización sin calidad de datos o sin una cultura organizacional adecuada puede limitar el impacto real de la tecnología.*

Tendencias para 2026 y Transformación Operativa

En 2026, las bases de datos evolucionan hacia ecosistemas integrados con IA generativa, automatizando pipelines y clasificando datos. Tendencias clave incluyen analytics everywhere (democratización del análisis donde el 80% de los empleados accede a dashboards), data fabric (integración automática de datos heterogéneos), y decision intelligence (IA que no solo analiza sino recomienda acciones). Las bases de datos en la nube (Snowflake, BigQuery, Amazon Redshift) permiten escalabilidad elástica, pagando solo por lo que se consume.

En el contexto de retail, la integración de IA con bases de datos transaccionales permite recomendaciones en tiempo real durante el proceso de compra, predicción de demanda para optimizar inventario, y detección automática de patrones anómalos que podrían indicar fraude o cambios en el comportamiento del consumidor.

Implementación de una Cultura de Datos

Para maximizar el impacto, las organizaciones deben fomentar una cultura de datos, donde los datos se convierten en el centro de la estrategia. Pasos clave incluyen: definir una estrategia con compromiso ejecutivo, invertir en infraestructura y talento, democratizar el acceso a datos con herramientas de autoservicio, medir resultados con KPIs claros, y iterar continuamente sobre las soluciones implementadas.

Las bases de datos no solo optimizan operaciones, sino que redefinen la competitividad. Estudios de la industria indican que cada dólar invertido en business intelligence genera retornos significativos en eficiencia operativa, reducción de costos y mejora de ingresos.

Data Warehouse

Los procesos de extracción, transformación y carga de datos, mejor conocidos como ETL por sus siglas en inglés (Extract, Transform, Load), se enmarcan en las actividades clave en el contexto de las bases de datos corporativas y los almacenes de datos. Un data warehouse es un repositorio centralizado que integra datos de múltiples fuentes operacionales para facilitar el análisis y la toma de decisiones estratégicas.

Dentro de las fases de la construcción de un data warehouse, el ETL es una de las tareas con mayor costo, tanto por tiempo como por recursos, estando esta labor asociada a la unificación de datos provenientes de diferentes fuentes, los cuales deberán ser limpiados y estructurados bajo un esquema común antes de ser cargados.

Uno de los aspectos importantes a considerar en un sistema para la administración de datos es el almacenamiento; dado que de esto dependen factores como acceso, disponibilidad, escalabilidad, seguridad, rendimiento y costo. El data warehouse está diseñado para almacenar datos históricos de manera optimizada para consultas analíticas, a diferencia de las bases de datos operacionales que priorizan las transacciones rápidas.

Los data warehouse están modelados usualmente por una estructura de base de datos multidimensional, donde se tiene una serie de atributos agrupados en dimensiones que hacen parte de una tabla de hechos central. Las arquitecturas incluyen la estructura en estrella (tabla de hechos rodeada de dimensiones) y la estructura copo de nieve (dimensiones normalizadas para reducir redundancia).

En nuestro proyecto implementamos un proceso ETL completo: extrajimos datos de un archivo CSV (BigBasket Products.csv con 27,555 productos), los transformamos mediante filtros por categoría, precio y deduplicación, y los cargamos en PostgreSQL. Las 6 tareas clásicas del ETL se aplicaron:

Tarea 1 - Identificación de fuentes: El dataset BigBasket de Kaggle (fuente heterogénea en formato CSV).

Tarea 2 - Transformación: Filtrado por categorías, conversión de tipos, cálculo de valores derivados, limpieza de espacios y deduplicación.

Tarea 3 - Unión de fuentes: Consolidación en un único DataFrame de Pandas con 9,996 productos limpios.

Tarea 4 - Selección del destino: PostgreSQL corriendo en Docker, base de datos postgres, puerto 5432.

Tarea 5 - Unión de atributos: Mapeo de columnas del CSV a columnas de la tabla articulos en PostgreSQL.

Tarea 6 - Carga de datos: Inserción masiva con `pandas.to_sql()` usando `chunksize=10000` y `method='multi'` para optimizar rendimiento.

Relación con Data Warehouse

Una base de datos operacional (OLTP) y un data warehouse (OLAP) están estrechamente relacionados, pero cumplen propósitos distintos y se complementan en la cadena de valor de los datos.

Definiciones

Base de datos (OLTP): Sistema para almacenar y gestionar datos actuales en transacciones rápidas, como registrar ventas o actualizaciones. Optimizada para lecturas/escrituras simples, con esquemas normalizados. En nuestro proyecto, las tablas ventas y detalle_ventas funcionan como el sistema transaccional.

Data warehouse (OLAP): Repositorio centralizado para datos históricos integrados de múltiples fuentes, optimizado para consultas complejas y reporting. Nuestras consultas de itemsets (JOIN entre detalle_ventas y articulos agrupando por id_venta y contando categorías) son consultas típicas de OLAP.

***ANEXO:** La evolución hacia arquitecturas modernas como lakehouse y data mesh responde a la necesidad de integrar análisis avanzado y escalabilidad, manteniendo gobierno y calidad de datos.*

Diferencias Clave

Característica	Base de Datos (OLTP)	Data Warehouse (OLAP)
Propósito	Transacciones operativas diarias	Análisis y reportes estratégicos
Datos	Actuales, en tiempo real	Históricos, integrados
Consultas	Simples, rápidas (SELECT por ID)	Complejas, agregaciones (GROUP BY, JOIN)
Esquema	Normalizado (3FN)	Desnormalizado (estrella, copo de nieve)
Usuarios	Operativos (cajeros, vendedores)	Analíticos (gerentes, científicos de datos)
Volumen	GB a TB	TB a PB
Actualización	Continua (INSERT/UPDATE)	Periódica (ETL batch o streaming)

Relación Principal y Funcionamiento

El data warehouse se alimenta de bases de datos operacionales mediante ETL: Extract (extraer datos de fuentes como ERP, CRM, APIs), Transform (limpiar, integrar, enriquecer los datos) y Load (cargar al warehouse). Esto crea una vista unificada sin afectar el rendimiento de los sistemas operacionales. En nuestro caso, el script `generar_transacciones.py` extrae datos de la tabla `articulos`, genera transacciones sintéticas, y las carga en las tablas `ventas`, `detalle_ventas` y `transacciones`.

Proceso ETL

Extract: De fuentes como archivos CSV, APIs, bases de datos operacionales. En nuestro proyecto: `pd.read_csv('BigBasket Products.csv')` y `pd.read_sql('SELECT * FROM articulos', engine)`.

Transform: Limpieza (eliminar duplicados con `drop_duplicates`), normalización (`str.strip()` para espacios), filtrado (máscaras booleanas por categoría), agregaciones (muestreo estratificado con `groupby`).

Load: Carga masiva con `df.to_sql('articulos', engine, if_exists='replace', chunksize=10000, method='multi')`. TRUNCATE para limpiar antes de recargar.

Beneficios de la Relación

La integración OLTP-OLAP permite análisis avanzados sin sobrecargar los sistemas transaccionales, mejora la calidad de datos mediante transformaciones ETL, y soporta técnicas de IA y Machine Learning. En nuestro proyecto, las consultas analíticas de itemsets (que involucran JOINS sobre millones de registros) se ejecutan sobre las tablas cargadas sin afectar la generación de nuevas transacciones.

Tendencias Modernas

- Lakehouse: Combina las ventajas de data lakes (almacenamiento barato de datos crudos) con data warehouses (consultas estructuradas). Tecnologías como Delta Lake proporcionan transacciones ACID sobre almacenamiento de objetos.
- Real-time warehousing: Ingesta de datos mediante streaming (Kafka, Kinesis) en lugar de ETL batch, permitiendo análisis en tiempo real.
- IA integrada: AutoML para generación automática de modelos y consultas en lenguaje natural que se traducen a SQL.

Ejemplos Prácticos

Banca: OLTP para procesar transacciones bancarias en tiempo real, OLAP para análisis de fraude histórico y detección de patrones sospechosos.

Retail: OLTP para registrar ventas diarias en punto de venta, OLAP para identificar tendencias estacionales y patrones de compra. Nuestro proyecto implementa exactamente este escenario: las tablas ventas/detalle_ventas simulan el OLTP, y los scripts de itemsets/reglas de asociación funcionan como consultas OLAP.

Problema de la Canasta Básica

Introducción

El análisis de canasta básica (Market Basket Analysis o MBA) es una técnica de análisis de datos que se utiliza para descubrir las asociaciones entre los productos comprados por los clientes (Santa Cruz, 2013; Gravitator, 2025). Se basa en la idea de que ciertos productos tienden a ser comprados juntos con mayor frecuencia, y al identificar estas relaciones, las empresas pueden tomar decisiones estratégicas más informadas.

Por ejemplo, es posible que los clientes que compran frutas y verduras también tiendan a comprar lácteos y panadería, o que quienes compran productos de limpieza también adquieran artículos de higiene personal. Estas asociaciones revelan patrones interesantes que pueden aprovecharse para optimizar la disposición de productos en estantes, diseñar promociones cruzadas y mejorar la experiencia del cliente.

Métricas Fundamentales

El análisis utiliza tres métricas principales para evaluar las asociaciones entre productos:

1. Soporte (Support)

Mide la frecuencia con que aparece un conjunto de productos en el conjunto de datos, calculándose dividiendo el número de transacciones que contienen ese conjunto entre el total de transacciones. Indica qué tan popular es una combinación de productos. En nuestro proyecto, con 400,000 ventas, un soporte de 0.85 para Fruits & Vegetables significa que 340,000 ventas incluyen esa categoría.

2. Confianza (Confidence)

Mide la probabilidad de que el producto B se compre cuando se adquiere el producto A, calculándose dividiendo las transacciones que contienen ambos productos entre las que solo contienen el producto A. En nuestro análisis, una confianza de 0.92 para Bakery → Fruits/Veg significa que el 92% de las ventas con Bakery también incluyen Fruits & Vegetables.

3. Lift (Elevación)

Compara la co-ocurrencia observada de dos productos con su co-ocurrencia esperada si fueran independientes, dividiéndose el soporte del par de productos entre el producto de sus soportes individuales. Un lift mayor a 1 indica asociación positiva (se compran juntos más de lo esperado por azar), igual a 1 indica independencia, y menor a 1 indica asociación negativa. En nuestros resultados, la regla Baby → Gourmet presentó el lift más alto (~1.14).

Etapas

El proceso para obtener resultados se puede dividir en 6 etapas:

Preparación de los datos: Recopilar los datos de las transacciones. Cada transacción se representa como un conjunto de productos o categorías. En nuestro caso, consultamos `detalle_ventas JOIN articulos` desde PostgreSQL para obtener las categorías presentes en cada una de las 400,000 ventas. Para cada venta se genera un frozenset de categorías únicas.

Cálculo de medidas: Se calcula la frecuencia de co-ocurrencia de los conjuntos de categorías. La medida principal es cuántas ventas contienen cada combinación posible, desde categorías individuales (itemset 1) hasta quintetos (itemset 5).

Generación de conjuntos frecuentes: Utilizando `itertools.combinations()` se generan todas las combinaciones posibles: 10 individuales, 45 pares, 120 tríos, 210 cuartetos y 252 quintetos (637 combinaciones totales). Se verifica el soporte de cada una contra las 400,000 ventas.

Generación de reglas de asociación: A partir de los conjuntos frecuentes, se generan reglas $X \rightarrow Y$. Una regla se compone de un antecedente (categoría X) y un consecuente (categoría Y). Se calculan las 90 reglas posibles (45 pares $\times$ 2 direcciones) con sus métricas de soporte, confianza y lift.

Evaluación y selección de reglas: Las reglas se evalúan utilizando el lift como métrica principal. Se ordenan de mayor a menor lift y se generan visualizaciones: gráfica de barras (top 20 por lift), heatmap de confianza (matriz 10 $\times$ 10) y scatter de soporte vs confianza (tamaño = lift).

Interpretación de los resultados: Las reglas seleccionadas revelan las asociaciones y patrones entre categorías de productos. En nuestro análisis, se encontró que las categorías con menor frecuencia individual (Baby Care, Gourmet) generan las reglas con mayor lift cuando aparecen juntas.

Algoritmo Apriori

Técnica fundamental de minería de datos que se utiliza principalmente para extraer conjuntos de elementos frecuentes y generar reglas de asociación. Funciona según el principio de identificar las relaciones entre ítems en grandes conjuntos de datos transaccionales.

Principio Fundamental

El algoritmo se basa en el principio "a priori": cualquier subconjunto de un conjunto de artículos frecuente debe ser frecuente. Esto significa que si un conjunto de elementos $\{A, B\}$ es frecuente, entonces $\{A\}$ y $\{B\}$ también deben serlo. Este principio permite podar el espacio de búsqueda eficientemente.

Proceso Paso a Paso

El algoritmo Apriori es iterativo: en cada ciclo el objetivo es encontrar los conjuntos candidatos para formar las reglas de asociación.

Paso 1 - Identificar Ítems Frecuentes Individuales: Se cuenta el número de ocurrencias de cada ítem escaneando la base una primera vez. Solo se mantienen aquellos ítems cuyo soporte supere el umbral mínimo. En nuestro proyecto, las 10 categorías de canasta básica son los ítems base y todas superan el umbral.

Paso 2 - Unión (Join): Se generan $(K+1)$ conjuntos a partir de K -conjuntos uniéndolos entre sí. Los conjuntos frecuentes del nivel anterior se combinan para formar candidatos del siguiente nivel. De 10 categorías individuales se generan 45 pares, luego 120 tríos, 210 cuartetos y 252 quintetos.

Paso 3 - Poda (Prune): Se verifica el soporte de cada candidato contra la base de datos. Si el candidato no cumple con el soporte mínimo, se considera poco frecuente y se elimina para reducir el espacio de búsqueda.

Paso 4 - Repetición: El algoritmo repite los pasos de unión y poda hasta que no se puedan generar más conjuntos frecuentes de mayor tamaño. En nuestro caso, hasta nivel 5.

Paso 5 - Generación de Reglas: Se determinan las reglas de asociación a partir de los k-itemsets encontrados, se cuentan y se determinan sus factores de soporte, confianza y lift.

Algoritmos Alternativos al Apriori

FP-Growth (Frequent Pattern Growth): Es el algoritmo más rápido de los tres principales (Apriori, ECLAT y FP-Growth), mientras que Apriori genera las reglas de asociación más completas y de alta calidad (Heaton, 2017).

Características principales de FP-Growth:

- Utiliza una estructura de árbol extendida llamada árbol de patrones frecuentes (FP-tree) que almacena información comprimida y crucial sobre patrones frecuentes.
- Solo escanea la base de datos dos veces (vs. múltiples escaneos de Apriori).
- No genera conjuntos candidatos, trabaja directamente con el árbol.
- Ideal para datasets muy grandes por su menor uso de memoria.

Ventajas:

- Mucho más rápido en conjuntos de datos grandes.
- Menor uso de memoria durante el procesamiento.
- Mayor rendimiento en tiempo de ejecución para todos los niveles de grupos de productos.

Desventajas:

- Más complejo de implementar.
- Apriori es mejor en términos de generar buenos candidatos en comparación con FP-Growth.

ECLAT (Equivalence Class Clustering and Bottom-up Lattice Traversal)

ECLAT es una versión más eficiente y escalable del algoritmo Apriori que trabaja de manera vertical, similar a la búsqueda en profundidad (DFS) de un grafo.

Características:

- Enfoque vertical vs. horizontal de Apriori.
- Comienza en la raíz del árbol, explora nodos del siguiente nivel de profundidad y sigue bajando hasta llegar al primer nodo terminal.
- Menos escaneos de la base de datos que Apriori.

Aplicaciones Prácticas

En Retail Físico:

a) Optimización de Planogramas: El MBA arroja información valiosa sobre qué productos posicionar en conjunto para incentivar compras cruzadas. En nuestro proyecto, los itemsets de nivel 2 revelan que Fruits & Vegetables y Bakery se compran juntos frecuentemente, sugiriendo que deberían estar en áreas cercanas del supermercado. Estrategias específicas incluyen identificar productos que generan tráfico y ubicarlos estratégicamente, colocar productos complementarios adyacentes a productos principales.

b) Gestión de Inventario Inteligente: Al conocer las combinaciones de productos más frecuentes, las empresas pueden evitar la falta de stock o el exceso de inventario, mejorando la eficiencia y reduciendo costos.

c) Efectividad de Promociones: El MBA ayuda a entender cómo responden los clientes a las promociones, permitiendo evaluar la efectividad comparativa de descuentos directos, combos y promociones cruzadas.

En E-commerce:

Sistemas de recomendación en tiempo real: Sugerencias dinámicas tipo 'También te puede gustar', ofertas personalizadas durante el checkout, y cross-selling automatizado basado en patrones históricos. Nuestro Ejercicio 3 (KNN Hamming) implementa este caso de uso: recomienda categorías que el vecino más similar compra pero el cliente aún no.

Aplicaciones en Otros Sectores: MBA tiene aplicaciones en banca (detectar transacciones fraudulentas reconociendo patrones inusuales), telecomunicaciones (analizar deserción de clientes identificando patrones de cancelación), salud (identificar comorbilidades y asociaciones entre síntomas), seguros (detectar reclamaciones fraudulentas) y finanzas (detección de patrones en transacciones).

Herramientas y Tecnologías

El análisis se puede implementar con diversas herramientas:

Python: Bibliotecas como mlxtend, pandas, numpy, itertools. En nuestro proyecto usamos pandas para consultar la BD, itertools.combinations para generar itemsets, y matplotlib para visualizaciones.

R: Paquetes arules, arulesViz para visualización avanzada de reglas de asociación.

Plataformas comerciales: SAS, IBM SPSS, Microsoft Analysis Services.

Herramientas especializadas: Alteryx, software de BI como Tableau y Power BI.

Limitaciones y Desafíos Críticos

Problemas de Datos Dispersos (Sparse Data)

Uno de los principales problemas en Market Basket Analysis es lidiar con datos dispersos. En muchos casos, las transacciones tienen pocos artículos y solo observamos un pequeño subconjunto de posibles combinaciones. Para mitigar esto en nuestro proyecto, generamos transacciones con

10-25 productos y mínimo 5 categorías diferentes por ticket, asegurando densidad suficiente para encontrar patrones significativos.

Exceso de Reglas Obvias

Los conjuntos de datos transaccionales típicos pueden soportar cientos o miles de reglas de asociación obvias por cada regla interesante, y filtrar las reglas es una tarea no trivial. No existe una medida única acordada y diferentes medidas pueden resultar en clasificaciones muy diferentes de reglas de asociación. Por eso utilizamos el lift como métrica principal de filtrado.

Interpretación y Causalidad

Las reglas de asociación solo identifican correlaciones, no causalidad. Si pan y leche se compran juntos frecuentemente, ¿es porque el pan hace que la gente compre leche, o simplemente son productos básicos que todos necesitan? Es importante ser cauteloso al tomar decisiones basadas en estos hallazgos.

Naturaleza Estática del Análisis

MBA a menudo asume que los patrones de compra permanecen constantes en el tiempo, lo cual no siempre es el caso. El comportamiento del consumidor puede cambiar debido a temporadas, tendencias o condiciones económicas. Por ejemplo, durante la temporada navideña, combinaciones inusuales de artículos podrían comprarse juntos como regalos, lo que no sería indicativo de patrones regulares de compra.

Problemas de Escalabilidad

A medida que crece el tamaño del conjunto de datos, la complejidad computacional aumenta exponencialmente. Una cadena de supermercados con millones de transacciones por día y miles de productos genera un espacio de combinaciones masivo. En nuestro proyecto con 10 categorías el espacio es manejable (637 combinaciones), pero con productos individuales (9,996) sería computacionalmente prohibitivo sin algoritmos optimizados como FP-Growth.

Calidad de Datos: La precisión de los resultados depende de la calidad de los datos utilizados. Problemas comunes incluyen datos faltantes, registros duplicados, nombres de productos inconsistentes y errores de captura. Por eso dedicamos un paso completo (Paso 4 de Limpieza.py) a la limpieza y deduplicación del dataset.

DEFINICIÓN DEL PROYECTO

Este proyecto es un sistema de base de datos para el análisis de canasta básica de un supermercado simulado. Combina datos reales de productos (del catálogo BigBasket de Kaggle con 27,555 productos) con transacciones generadas sintéticamente para crear un entorno realista de alto volumen de información sobre el cual aplicar técnicas de minería de datos y aprendizaje automático.

El objetivo es construir una base de datos con:

- 9,996 productos de canasta básica en el catálogo (filtrados de 27,555 originales)
- 400,000 ventas registradas
- Aproximadamente 6.8 millones de líneas de detalle de ventas (10-25 productos por venta)
- 400,000 transacciones financieras (relación 1:1 con ventas)

Todo se almacena en PostgreSQL bajo un esquema de 4 tablas relacionadas, simulando el backend de un supermercado.

Dataset: <https://www.kaggle.com/datasets/surajjha101/bigbasket-entire-product-list-28k-datapoints>

Problemas Al Usar El Dataset

El proyecto inició con un dataset de PROFECO (México) contenido en un PDF de 250 páginas con 4,004 registros. Este dataset presentó tres problemas graves: primero, los productos estaban segmentados por estado (Aguascalientes con 54% de los registros), haciendo imposible generar transacciones coherentes ya que un cliente no compra productos de diferentes estados. Segundo, el 43% eran artículos irrelevantes (medicamentos como Tramadol, juguetes LEGO, electrónicos como laptops). Tercero, solo tenía 624 productos únicos, insuficiente para los 10,000 requeridos.

El dataset BigBasket también requirió limpieza significativa. De las 11 categorías originales, 4 mezclaban productos relevantes e irrelevantes: Beauty & Hygiene (7,867 productos donde solo 2,685 eran higiene esencial, excluyendo cosméticos y maquillaje), Cleaning & Household (2,675 donde solo 1,525 eran limpieza básica, excluyendo papelería y artículos religiosos), Baby Care (610 donde solo 501 eran esenciales) y Gourmet & World Food (4,690 donde solo 2,475 eran alimentos básicos, excluyendo snacks gourmet de lujo).

Adicionalmente, se encontraron 608 productos duplicados (mismo nombre, marca y precio), 97 con precios fuera de rango (>2,000 rupias), productos sin nombre o marca, y 8,626 ratings nulos (31%) que fueron normalizados a 0. Los tipos de dato requerían conversión: `sale_price` y `market_price` se convirtieron a numérico con `pd.to_numeric(errors='coerce')`, y las columnas de texto se limpiaron con `str.strip()`.

Herramientas y Tecnologías Utilizadas

Tecnología	Versión	Propósito en el proyecto
Python	3.13	Lenguaje principal para ETL, generación de datos sintéticos, análisis de itemsets, reglas de asociación, KNN y visualización
Pandas	2.x	Carga de CSV con read_csv(), transformación de DataFrames, carga masiva a PostgreSQL con to_sql()
NumPy	1.x	Generación aleatoria con semillas (seed=42), cálculos de distancia Euclidiana y Hamming, distribución multinomial para pesos por categoría
PostgreSQL	Docker	Base de datos relacional con 4 tablas, PRIMARY KEY, FOREIGN KEY, SERIAL, TRUNCATE, ALTER TABLE
SQLAlchemy	2.x	ORM para conexión Python-PostgreSQL mediante create_engine() y text() para queries SQL
psycopg2	2.x	Driver nativo de PostgreSQL para Python, requerido por SQLAlchemy
Matplotlib	3.x	Generación de 14 gráficas PNG: barras horizontales, heatmaps, scatter plots, paneles de recomendación
itertools	Estándar	Módulo de Python para generar combinaciones de categorías (combinations) en el cálculo de itemsets
Docker	Latest	Contenedor para ejecutar PostgreSQL sin instalación nativa, facilita portabilidad y configuración
PyCharm	2025+	IDE de desarrollo con integración a base de datos, consola SQL, panel de tablas y depurador
HTML/CSS/JS	Estándar	Dashboard web con pestañas para presentar los 3 ejercicios, imágenes embebidas en Base64

Arquitectura de la Base de Datos

La base de datos contiene 4 tablas principales con relaciones de integridad referencial:

Tabla	Registros	Descripción y columnas
articulos	9,996	Catálogo: id_articulo (PK), product, category, sub_category, brand, sale_price, market_price, type, rating
ventas	400,000	Resumen por ticket: id_venta (PK SERIAL), id_cliente, fecha, hora, cantidad_productos, total_venta
detalle_ventas	~6.8M	Desglose: id_detalle (PK SERIAL), id_venta (FK), id_articulo (FK), cantidad, precio_unit, subtotal
transacciones	400,000	Financiero: id_transaccion (PK SERIAL), id_venta (FK), id_cliente, fecha, hora, cantidad_productos, total_pagado

Explicación de los Scripts

Limpieza.py

Punto de entrada del proyecto. Este script:

- Carga el archivo BigBasket Products.csv con Pandas (27,555 productos)
- Limpia espacios en blanco con `str.strip()` en columnas de texto
- Aplica filtro de dos niveles: 6 categorías completas + 4 categorías con subcategorías específicas
- Filtra por precio (`sale_price > 0` y `≤ 2000`)
- Elimina duplicados por (`product, brand, sale_price`)
- Aplica muestreo estratificado a ~10,000 productos
- Crea las 4 tablas en PostgreSQL con `DROP TABLE IF EXISTS CASCADE`
- Carga los artículos con `to_sql()` y agrega PRIMARY KEY con `ALTER TABLE`

Concepto clave: ETL (Extract, Transform, Load). Este script implementa el patrón ETL clásico: extrae datos del CSV, los transforma (5 pasos de limpieza) y los carga en la base de datos.

generar_transacciones.py

El script más complejo. Genera tres tablas relacionadas simultáneamente:

- ventas: 400,000 registros, uno por ticket, con cliente, fecha, hora, cantidad y total
- detalle_ventas: ~6.8 millones de registros, 10-25 por ticket, con producto, cantidad, precio y subtotal
- transacciones: 400,000 registros, relación 1:1 con ventas, registro financiero

Cada venta selecciona productos aleatorios del catálogo usando pesos por categoría (Fruits & Vegetables peso 5.0 vs Baby Care peso 0.8), asegurando mínimo 5 categorías diferentes. Las fechas se generan entre enero y diciembre 2024, con distribución horaria realista (picos a las 10-12 y 17-19). Se aplica descuento aleatorio del 0-10% sobre `sale_price`.

itemsets.py

Calcula los itemsets frecuentes de nivel 1 a 5 consultando `detalle_ventas JOIN articulos`. Para cada venta obtiene el conjunto de categorías únicas, luego evalúa las 637 combinaciones posibles ($10+45+120+210+252$) contando cuántas ventas contienen cada combinación. Genera 6 gráficas PNG (una por nivel + resumen) y 6 archivos CSV.

ejercicio1_reglas_asociacion.py

Calcula soporte, confianza y lift para las 90 reglas de asociación entre pares de categorías. Muestra un ejemplo paso a paso idéntico al formato de libreta. Genera 3 gráficas: barras de lift con línea en `lift=1`, heatmap de confianza 10×10, y scatter de soporte vs confianza donde tamaño = lift.

ejercicio2_knn_euclidiana.py

Consulta 200 clientes reales de la BD con su gasto promedio y productos promedio. Los clasifica en Gasto Alto/Bajo usando la mediana. Dado un nuevo cliente, calcula la distancia Euclidiana a todos y clasifica por votación de los K=3 más cercanos. Muestra el cálculo paso a paso con fórmula. Genera 2 gráficas.

ejercicio3_knn_hamming.py

Representa ventas como vectores binarios de 10 dimensiones (1 por categoría). Selecciona una venta X con 5-6 categorías y vecinos con 7-9 para que haya diferencias reales. Calcula distancia de Hamming (posiciones diferentes) y con K=1 recomienda las categorías que el vecino tiene pero X no. Genera 3 gráficas incluyendo el panel de recomendación.

Flujo de Ejecución del Proyecto

1. Limpieza.py — Crea tablas y carga 9,996 artículos
2. generar_transacciones.py — Genera y carga 400K ventas + ~6.8M detalles + 400K transacciones
3. itemsets.py — Calcula 637 itemsets y genera 6 gráficas + 6 CSVs
4. ejercicio1_reglas_asociacion.py — 90 reglas con 3 gráficas
5. ejercicio2_knn_euclidiana.py — Clasificación KNN con 2 gráficas
6. ejercicio3_knn_hamming.py — Recomendación KNN con 3 gráficas

Conceptos Clave Que Demuestra El Proyecto

Generación de Datos Sintéticos

Cuando no se tienen suficientes datos reales, se generan datos sintéticos (artificiales pero realistas). Se usan semillas de aleatoriedad (`np.random.seed(42)`) para garantizar reproducibilidad. Los pesos por categoría simulan comportamiento real donde frutas y lácteos se compran más que gourmet o baby care.

Relaciones Entre Tablas (Esquema Relacional)

El proyecto implementa un esquema relacional donde `detalle_ventas` es la tabla central que se relaciona con `articulos` (FK `id_articulo`) y `ventas` (FK `id_venta`). Esto permite consultas tipo: ¿cuántas ventas incluyeron productos de Bakery y Beverages simultáneamente?

Carga Masiva de Datos (Bulk Load)

Subir millones de registros uno por uno sería extremadamente lento. Por eso se usa `pandas.to_sql()` con `chunksize=10000` y `method='multi'`, lo cual optimiza drásticamente el rendimiento. La tabla `detalle_ventas` (~6.8M registros) se carga en minutos.

Resumen General Del Proyecto

Este proyecto consiste en la construcción de una base de datos analítica para un supermercado simulado, diseñada como un entorno realista para la aplicación de minería de datos y aprendizaje automático. La solución parte de un catálogo real de 27,555 productos de BigBasket (Kaggle), que mediante un proceso de ETL de 5 pasos se reduce a 9,996 artículos exclusivamente de canasta básica distribuidos en 10 categorías, 64 subcategorías y 1,362 marcas.

Sobre este catálogo se generan 400,000 transacciones sintéticas con características realistas: cada compra incluye entre 10 y 25 productos de al menos 5 categorías diferentes, con pesos de probabilidad que reflejan frecuencias reales de compra, distribución horaria con picos matutinos y vespertinos, y descuentos aleatorios del 0-10%.

El sistema implementa un flujo ETL donde los datos son extraídos desde CSV, transformados mediante Pandas y cargados en PostgreSQL. Las 4 tablas (articulos, ventas, detalle_ventas, transacciones) suman aproximadamente 7.6 millones de registros con integridad referencial garantizada por claves foráneas.

Sobre esta base se ejecutan tres tipos de análisis presentados como casos de uso: (1) Reglas de asociación con soporte, confianza y lift entre 10 categorías, (2) Clasificación de clientes con KNN $K=3$ y distancia Euclidiana, y (3) Recomendación de categorías con KNN $K=1$ y distancia de Hamming. Cada análisis incluye visualizaciones profesionales, totalizando 14 gráficas PNG presentadas en un dashboard web con pestañas.

Desde el punto de vista técnico, el proyecto demuestra el uso integrado de Python 3.13, PostgreSQL en Docker, Pandas, NumPy, SQLAlchemy, Matplotlib y HTML/CSS/JavaScript, así como estrategias de carga masiva, generación de datos sintéticos con reproducibilidad, y presentación de resultados en formato web autocontenido.

Análisis de Atributos Faltantes para KNN y SVM

El proyecto cuenta actualmente con cuatro tablas principales que almacenan información sobre productos, ventas, detalles de transacciones y registros financieros. Sin embargo, para poder implementar dos de los algoritmos más importantes en aprendizaje supervisado — K-Nearest Neighbors (KNN) y Support Vector Machines (SVM) — es necesario evaluar si los datos existentes son suficientes o si se requieren atributos adicionales.

Tanto KNN como SVM son algoritmos de aprendizaje supervisado, lo que significa que para poder entrenarse necesitan dos elementos fundamentales: un conjunto de atributos numéricos o categóricos bien definidos (features) y una variable objetivo o etiqueta (target) que indique a qué clase pertenece cada registro. A continuación se analiza qué tiene la base de datos, qué necesitan estos algoritmos y qué hace falta.

Datos Disponibles en la Base de Datos

Antes de identificar lo que falta, es importante conocer con qué se cuenta actualmente:

Tabla	Atributos disponibles
articulos	id_articulo, product, category, sub_category, brand, sale_price, market_price, type, rating
ventas	id_venta, id_cliente, fecha, hora, cantidad_productos, total_venta
detalle_ventas	id_detalle, id_venta, id_articulo, cantidad, precio_unit, subtotal
transacciones	id_transaccion, id_venta, id_cliente, fecha, hora, cantidad_productos, total_pagado

KNN y SVM Necesitan Datos Especiales

Antes de listar los atributos faltantes, es clave entender cómo funcionan estos dos algoritmos y qué requieren:

K-Nearest Neighbors (KNN)

KNN es un algoritmo de clasificación que funciona calculando la distancia entre un nuevo punto de datos y los K puntos más cercanos en el conjunto de entrenamiento. La clase más frecuente entre esos K vecinos se asigna al nuevo punto. En nuestro proyecto implementamos KNN en dos variantes: con distancia Euclidiana (Ejercicio 2, para clasificar clientes por gasto) y con distancia de Hamming (Ejercicio 3, para recomendar categorías usando vectores binarios).

Para que KNN funcione correctamente necesita:

- Atributos numéricos o convertibles a números (no texto libre). En nuestro caso usamos `gasto_promedio` y `productos_promedio` (Ejercicio 2) y vectores binarios de 10 dimensiones (Ejercicio 3).

- Todos los atributos deben estar en escalas similares (normalización o estandarización), ya que las diferencias de escala entre variables distorsionan el cálculo de distancias. Si el gasto va de \$100 a \$10,000 pero los productos van de 10 a 25, el gasto dominará completamente la distancia.
- Una variable objetivo (etiqueta) bien definida que indique a qué categoría pertenece cada registro. En el Ejercicio 2 la etiqueta es 'Gasto Alto' / 'Gasto Bajo', derivada de la mediana del gasto.
- Una cantidad razonable de datos por clase para que los vecinos sean representativos.

Máquinas de Soporte Vectorial (SVM)

SVM es un algoritmo que busca el hiperplano óptimo que separa dos o más clases en el espacio de características. Es especialmente poderoso para clasificación binaria y datos de alta dimensionalidad.

Para que SVM funcione correctamente necesita:

- Variables numéricas normalizadas, ya que SVM es extremadamente sensible a diferencias de escala entre atributos.
- Una variable objetivo binaria o multiclase claramente definida.
- Datos sin valores nulos ni atributos de texto sin transformar.
- Suficientes ejemplos de cada clase para evitar hiperplanos sesgados.

Atributos Críticos Faltantes

A continuación se detallan los atributos que no están presentes en la base de datos y que serían necesarios para implementar KNN y SVM de manera más robusta:

Variable Objetivo o Etiqueta (Target Label)

Nivel de Prioridad: CRÍTICO - Sin este atributo, KNN y SVM no pueden implementarse en ninguna circunstancia.

Tanto KNN como SVM son algoritmos de aprendizaje supervisado, lo que significa que aprenden a partir de ejemplos etiquetados. Sin una variable que diga 'este cliente es de Gasto Alto', 'esta transacción es anómala' o 'este producto pertenece a tal segmento', los algoritmos no tienen nada que aprender ni predecir.

En nuestro proyecto resolvimos esto de la siguiente manera:

Ejercicio 2 (KNN Euclidiana): La etiqueta se genera dinámicamente calculando la mediana del gasto promedio de 200 clientes. Los que están por encima son 'Gasto Alto', los que están por debajo son 'Gasto Bajo'. Esto es una clasificación binaria derivada de los datos existentes.

Ejercicio 3 (KNN Hamming): No se usa una etiqueta de clase sino vectores binarios de 10 dimensiones (categorías compradas vs no compradas). El objetivo es recomendación, no clasificación.

Dependiendo del objetivo del negocio, la etiqueta podría ser:

Tipo_Cliente: Clasifica a cada cliente en categorías como VIP, Regular o Nuevo, basándose en su historial de compras. Permite personalizar estrategias de marketing.

Fraude (Sí/No): Identifica transacciones sospechosas o anómalas. Útil para prevención de pérdidas.

Churn (Se fue/Se quedó): Predice si un cliente dejará de comprar. Fundamental para estrategias de retención.

Categoría_Producto: Clasifica cada producto en una familia. En nuestro caso ya existe como la columna category con 10 valores.

Atributos Demográficos del Cliente

Nivel de Prioridad: **CRÍTICO** para clasificación de clientes.

Actualmente la base de datos solo tiene id_cliente como identificador numérico. Esto equivale a tener el nombre de una persona sin saber nada sobre ella. Para que KNN o SVM puedan 'describir' a un cliente y encontrar patrones significativos, se beneficiarían de atributos como:

Edad: Permite identificar segmentos generacionales con comportamientos de compra distintos (jóvenes prefieren snacks y bebidas, adultos mayores priorizan alimentos básicos y medicamentos).

Género: Tiene correlación estadística con categorías de productos adquiridos (higiene personal, baby care).

Ubicación geográfica detallada: Ciudad o región para identificar patrones locales de consumo.

Nivel socioeconómico o segmento de ingreso: Influye directamente en el ticket promedio de compra y en las categorías preferidas.

Antigüedad como cliente: Cuántos meses o años lleva comprando en la tienda. Un cliente de 3 años tiene patrones más estables que uno de 1 mes.

Sin estos atributos, el algoritmo no tiene manera de encontrar 'vecinos similares' (KNN) ni de trazar fronteras de decisión (SVM) entre tipos de clientes basados en características intrínsecas. En nuestro proyecto compensamos esta carencia usando métricas derivadas del comportamiento de compra (gasto promedio, productos promedio).

Métricas RFM Precalculadas

Nivel de Prioridad: **IMPORTANTE** - Derivables mediante SQL, pero no disponibles directamente.

RFM es un modelo estándar de la industria retail para caracterizar el comportamiento de compra de un cliente. Las siglas significan:

Recencia (R): ¿Cuántos días han pasado desde la última compra del cliente? Un cliente que compró ayer es más valioso que uno que compró hace un año.

Frecuencia (F): ¿Cuántas veces ha comprado el cliente en un período determinado? La frecuencia alta indica fidelidad.

Valor Monetario (M): ¿Cuánto dinero ha gastado el cliente en total? Clientes de alto valor merecen estrategias diferenciadas.

Estas métricas son numéricas y continuas, lo que las hace perfectas como features para KNN y SVM. Aunque podrían calcularse con consultas SQL sobre las tablas existentes (usando MAX(fecha) para recencia, COUNT(*) para frecuencia y SUM(total_venta) para monto), no están disponibles como columnas precalculadas. En nuestro Ejercicio 2, usamos AVG(total_venta) y AVG(cantidad_productos) como aproximación a las métricas RFM.

Categoría Estructurada de Producto

Nivel de Prioridad: CRÍTICO - El texto libre no puede usarse directamente.

La columna product en artículos contiene cadenas de texto como 'Garlic Oil - Vegetarian Capsule 500 mg' o 'Water Bottle - Orange'. Aunque esta información es comprensible para un humano, los algoritmos de ML no pueden procesar texto libre directamente porque no pueden calcular distancias entre textos sin una transformación previa, el espacio de valores posibles es prácticamente infinito, y dos descripciones muy similares pueden interpretarse como completamente distintas.

En nuestro proyecto esto ya está resuelto: la columna category con 10 valores discretos ('Beverages', 'Bakery, Cakes & Dairy', etc.) proporciona la categorización estructurada necesaria. Adicionalmente, sub_category con 64 valores ofrece un nivel de granularidad más fino. Para el Ejercicio 3 (KNN Hamming), estas categorías se convierten en vectores binarios de 10 dimensiones, una representación ideal para algoritmos basados en distancia.

Variables Numéricas Normalizadas

Nivel de Prioridad: RECOMENDABLE - Especialmente crítico para SVM.

Los atributos numéricos actuales tienen escalas muy distintas entre sí: sale_price varía de 2.45 a 2,000, la cantidad de 1 a 3, el total de una venta puede ir de cientos a miles. Esta disparidad de escalas causa problemas serios:

En KNN: Las distancias calculadas estarán dominadas por la variable de mayor magnitud (precio), ignorando las demás. Un cliente que gasta \$5,000 vs \$4,900 (\$100 de diferencia) parecerá más diferente que uno que compra 10 vs 25 productos (15 de diferencia), cuando en realidad la diferencia de productos puede ser más significativa.

En SVM: El algoritmo tendrá dificultades para encontrar el hiperplano óptimo, ya que las variables con valores grandes ejercen más influencia sobre el margen de separación.

La solución es aplicar normalización Min-Max (escala de 0 a 1) o estandarización Z-Score (media 0, desviación estándar 1) a todos los atributos numéricos antes de entrenar el modelo. Esto puede hacerse fácilmente con scikit-learn (StandardScaler o MinMaxScaler) o manualmente en la consulta SQL.

Indicador de Devoluciones o Cancelaciones

Nivel de Prioridad: RECOMENDABLE - Enriquece el perfil del cliente.

Actualmente la base de datos no registra devoluciones ni cancelaciones. Contar con este atributo sería valioso porque un cliente con alta tasa de devoluciones tiene un perfil de riesgo diferente al de uno que nunca devuelve, y puede usarse como feature para detectar comportamientos anómalos o

fraudulentos. En nuestro proyecto sintético no se generaron devoluciones, pero en un entorno real sería un atributo importante.

Resumen Ejecutivo de Atributos Faltantes

Atributo	Prioridad	Estado	Solución aplicada
Variable Objetivo	CRÍTICO	Resuelto	Mediana del gasto
Demográficos	CRÍTICO	No disponible	Compensado con RFM
Métricas RFM	IMPORTANTE	Derivable	AVG(gasto), AVG(prod)
Categoría Producto	CRÍTICO	Resuelto	Columna category (10)
Normalización	RECOMENDABLE	Parcial	Escalas similares en E2
Devoluciones	RECOMENDABLE	No disponible	No aplica (sintético)

Comentarios

El análisis revela que la base de datos, tal como está configurada, permite implementar KNN en sus dos variantes (Euclidiana y Hamming) gracias a que se derivaron variables objetivo y features numéricos a partir de los datos transaccionales existentes. Los atributos críticos (variable objetivo y categoría de producto) están resueltos. Los atributos demográficos mejorarían la precisión pero no son bloqueantes. Para una implementación de SVM, se recomendaría agregar normalización explícita de las variables numéricas.

Resumen de Implementación

Resumen Ejecutivo

Se construyó la base de datos completa para que cumpliera con todos los requisitos establecidos en el marco teórico del problema de la canasta básica. Se depuraron las tablas con datos inconsistentes mediante un proceso ETL de 5 pasos, se generaron 400,000 transacciones sintéticas con diversidad de categorías, se calcularon itemsets frecuentes de nivel 1 a 5 (637 combinaciones), se generaron 90 reglas de asociación con soporte, confianza y lift, y se implementaron 3 ejercicios de Machine Learning como casos de uso.

El resultado es una base de datos que soporta: análisis de canasta básica con itemsets frecuentes, reglas de asociación, clasificación supervisada con KNN (distancia Euclidiana), y recomendación de productos con KNN (distancia de Hamming). Todo visualizado en un dashboard web con pestañas.

Cumplimiento de las 6 Etapas del Market Basket Analysis

El marco teórico define 6 etapas para el análisis de canasta básica. Estado de cada una:

#	Eta	Estado	Implementación
1	Preparación de datos	Completa	detalle_ventas JOIN articulos, frozenset de categorías por venta
2	Cálculo de medidas	Completa	Soporte calculado para 637 combinaciones sobre 400K ventas
3	Conjuntos frecuentes	Completa	itertools.combinations niveles 1-5 (10+45+120+210+252)
4	Reglas de asociación	Completa	90 reglas $X \rightarrow Y$ con soporte, confianza y lift
5	Evaluación de reglas	Completa	Ordenamiento por lift, top 20, heatmap, scatter
6	Interpretación	Completa	Gráficas, dashboard web, documentación

Las Tres Métricas Fundamentales

Métrica	Fórmula	Implementación
Soporte	$\text{ventas_con_XY} / \text{total_ventas}$	issubset() sobre frozensets por venta
Confianza	$\text{Soporte}(X \cup Y) / \text{Soporte}(X)$	División de soportes precalculados
Lift	$\text{Confianza}(X \rightarrow Y) / \text{Soporte}(Y)$	División confianza / soporte individual

Requisitos para KNN

Requisito	Estado	Cómo se cumplió
Features numéricos	Cumplido	Ej2: gasto_prom, productos_prom. Ej3: vector binario 10 dims
Variable objetivo	Cumplido	Ej2: Gasto Alto/Bajo por mediana. Ej3: recomendación (sin clase)
Distancia definida	Cumplido	Ej2: Euclidiana $\sqrt{(x_1-x_2)^2+(y_1-y_2)^2}$. Ej3: Hamming (bits diferentes)
Datos suficientes	Cumplido	50,000 clientes, 400,000 ventas
K definido	Cumplido	Ej2: K=3 (votación). Ej3: K=1 (vecino más cercano)

Integridad Referencial Verificada

Las 4 tablas mantienen integridad referencial completa:

- articulos.id_articulo es PRIMARY KEY, referenciado por detalle_ventas.id_articulo (FOREIGN KEY).
- ventas.id_venta es PRIMARY KEY (SERIAL), referenciado por detalle_ventas.id_venta y transacciones.id_venta.

- No existen registros huérfanos: cada detalle apunta a una venta y un artículo válidos.
- TRUNCATE...CASCADE y DROP TABLE IF EXISTS...CASCADE garantizan limpieza completa al re-ejecutar.

Herramientas y Tecnologías Utilizadas

Herramienta	Sección del MT	Uso específico
Python + Pandas	Herramientas p.21	ETL, generación sintética, itemsets, reglas, KNN
PostgreSQL	Base de Datos p.1	SGBD relacional con 4 tablas, FK, PK, SERIAL
itertools	Apriori p.17	combinations() para generar itemsets nivel 1-5
Matplotlib	Interpretación p.16	14 gráficas PNG para visualización de resultados
HTML/CSS/JS	Presentación	Dashboard web con pestañas para los 3 casos de uso

Resultados de Itemsets Frecuentes

Resumen por Tamaño de Itemset

Nivel	Combinaciones	Con soporte > 0	Soporte máximo
Itemset 1	10	10 (100%)	~92% (Fruits/Veg)
Itemset 2	45	45 (100%)	~85% (top par)
Itemset 3	120	120 (100%)	~78% (top trío)
Itemset 4	210	210 (100%)	~65% (top cuarteto)
Itemset 5	252	252 (100%)	~50% (top quinteto)

Todas las 637 combinaciones tienen soporte mayor a 0, lo cual indica que la diversidad forzada de categorías por ticket (mínimo 5) genera datos densos donde todas las combinaciones de categorías aparecen en al menos algunas ventas. El soporte máximo disminuye naturalmente a medida que aumenta el nivel del itemset, ya que es menos probable que 5 categorías específicas coincidan todas en la misma venta.

Itemsets de Tamaño 1

Los 10 itemsets de tamaño 1 representan la frecuencia individual de cada categoría. Las categorías con mayor soporte son Fruits & Vegetables y Bakery, Cakes & Dairy, lo cual es coherente con los pesos de probabilidad asignados en la generación de transacciones (peso 5.0 y 4.5 respectivamente). Baby Care y Gourmet & World Food tienen el menor soporte individual, reflejando que son categorías de nicho compradas por segmentos específicos de clientes.

Itemsets de Tamaño 2

Las 45 combinaciones de pares revelan qué categorías se compran juntas con mayor frecuencia. Los pares con mayor soporte combinan las categorías más populares individualmente. Los pares más interesantes desde el punto de vista analítico son aquellos cuyo soporte conjunto supera lo esperado por el producto de sus soportes individuales ($\text{lift} > 1$).

Itemsets de Tamaño 3, 4 y 5

A medida que aumenta el nivel, las combinaciones se vuelven más específicas y el soporte disminuye. Sin embargo, con mínimo 5 categorías por ticket, incluso los quintetos mantienen soportes significativos (~45-50% para los más frecuentes). Esto demuestra que la estrategia de diversidad forzada fue exitosa para generar datos adecuados para itemsets de nivel alto.

Resultados de las Reglas de Asociación

Se calcularon 90 reglas de asociación (45 pares de categorías $\times$ 2 direcciones) con sus tres métricas. Los resultados principales son:

- La regla con mayor lift fue Baby Care $\rightarrow$ Gourmet (~1.14), indicando que cuando Baby Care aparece en una venta, la probabilidad de que Gourmet también esté presente supera en un 14% lo esperado por azar.
- Las reglas con mayor confianza involucran Fruits & Vegetables como consecuente (>90%), lo cual se explica porque esta categoría tiene el mayor soporte individual y aparece en la gran mayoría de ventas.
- Todas las reglas tienen lift cercano a 1 (rango ~0.98 a ~1.14), lo cual indica que las categorías son relativamente independientes entre sí. Esto es consistente con la generación aleatoria ponderada de transacciones.

Las 3 gráficas generadas (barras de lift, heatmap de confianza, scatter soporte vs confianza) permiten visualizar estos patrones de manera clara y son presentadas en la pestaña 1 del dashboard web.

Contexto del Análisis

El presente análisis se basa en los resultados obtenidos del Market Basket Analysis aplicado sobre 400,000 transacciones de la base de datos en PostgreSQL, así como en la clasificación de clientes mediante KNN y el sistema de recomendación con distancia de Hamming. A partir de los itemsets frecuentes (637 combinaciones de 1 a 5 categorías) y las 90 reglas de asociación encontradas, se identifican problemáticas reales del negocio y se proponen modelos de negocio basados en datos.

Datos Clave del Análisis

Métrica	Valor
Total de ventas analizadas	400,000
Productos en catálogo	9,996
Categorías	10
Subcategorías	64
Marcas únicas	1,362
Clientes únicos	50,000
Itemsets calculados	637 (niveles 1-5)
Reglas de asociación	90
Gráficas generadas	14 PNG

Problemáticas Identificadas

Categorías con baja frecuencia de compra (Riesgo de Merma)

Descripción: Baby Care y Gourmet & World Food tienen el soporte individual más bajo entre las 10 categorías. Esto significa que un porcentaje menor de clientes compra productos de estas categorías en cada visita. En un entorno real con productos perecederos, baja rotación genera mermas por caducidad.

Impacto: Los productos de estas categorías con baja rotación generan mermas por caducidad. El inventario se acumula y se pierde rentabilidad, especialmente en productos de Bakery, Cakes & Dairy y Fruits & Vegetables que son perecederos.

Solución propuesta: Implementar ofertas cruzadas basadas en los itemsets encontrados para impulsar estas categorías, ajustar el inventario dinámicamente según la demanda real, y crear paquetes promocionales que combinen categorías de baja rotación con las de alta rotación.

Falta de Estrategia de Cross-Selling entre Categorías Complementarias

Descripción: Los itemsets de tamaño 2 revelan combinaciones frecuentes como Bakery+Beverages, Fruits+Foodgrains, Eggs+Cleaning, pero sin una estrategia activa de cross-selling estas oportunidades se desperdician.

Impacto: Se pierde oportunidad de incrementar el ticket promedio. Un cliente que compra Bakery tiene alta probabilidad de comprar Beverages, pero si no se le sugiere, la venta adicional puede no concretarse.

Solución propuesta: Implementar un sistema de recomendaciones (como nuestro Ejercicio 3 con KNN Hamming) que sugiera productos complementarios basados en las reglas de asociación con mayor soporte y lift.

Clientes sin Clasificación ni Estrategia Diferenciada

Descripción: Con 50,000 clientes, no todos tienen el mismo valor para el negocio. Sin una clasificación, se aplica el mismo trato a un cliente que gasta \$10,000 al mes y a uno que gasta \$500.

Impacto: Los clientes de alto valor no reciben el trato preferencial que merecen, y los de bajo valor no reciben estímulos para aumentar su gasto.

Solución propuesta: Implementar clasificación automática con KNN (como nuestro Ejercicio 2) que segmente clientes en Gasto Alto y Gasto Bajo, permitiendo estrategias diferenciadas: programa VIP para los de gasto alto, promociones de incremento para los de gasto bajo.

Ausencia de Sistema de Recomendación Personalizado

Descripción: Cada cliente recibe las mismas promociones genéricas sin importar su historial de compras. No se aprovecha la información de qué categorías compra cada cliente para sugerirle las que aún no ha explorado.

Impacto: Las promociones genéricas tienen menor tasa de conversión que las personalizadas. Un cliente que nunca ha comprado Gourmet no responderá igual a una promoción de Gourmet que uno que ya la conoce.

Solución propuesta: Usar KNN con distancia de Hamming (Ejercicio 3) para encontrar clientes similares y recomendar categorías que el vecino compra pero el cliente aún no. En nuestro ejercicio, esto se demostró exitosamente recomendando Eggs/Meat y Cleaning a un cliente que no las tenía.

Modelos de Negocio Basados en los Datos

Usando la clasificación de clientes (Gasto Alto/Bajo) y los patrones de compra descubiertos por los itemsets y reglas de asociación, se proponen los siguientes modelos de negocio:

Modelo: Programa de Lealtad por Niveles

Segmento objetivo: Todos los clientes (con beneficios escalonados).

Descripción: Crear un programa de puntos donde cada peso gastado acumula puntos canjeables. Los clientes de Gasto Alto obtienen multiplicador 3x, los de Gasto Bajo obtienen 1.5x durante su primer mes. Los puntos se canjean por descuentos en las categorías complementarias identificadas por las reglas de asociación. Por ejemplo, un cliente que acumula puntos comprando Bakery puede canjearlos en Beverages (par con alto soporte).

Beneficio esperado: Incrementar la frecuencia de compra en un 15-20% y reducir la tasa de inactividad al incentivar compras recurrentes.

Modelo: Suscripción Mensual de Canasta Básica

Segmento objetivo: Clientes de Gasto Alto con compras frecuentes.

Descripción: Ofrecer una suscripción mensual que entregue un paquete predefinido de productos de Foodgrains + Fruits & Vegetables + Bakery (las 3 categorías de mayor soporte individual). El paquete se personaliza según el historial del cliente usando las reglas de asociación. Si el cliente siempre compra Beverages con Bakery, se incluye automáticamente.

Beneficio esperado: Reducir merma de productos perecederos al tener demanda predecible, asegurar ingreso recurrente mensual, y mejorar la rotación de inventario.

Modelo: Combos Inteligentes por Itemsets

Segmento objetivo: Todos los clientes.

Descripción: Crear una sección de 'Combos Recomendados' generados automáticamente a partir de los itemsets de tamaño 3, 4 y 5. Ejemplo: Combo Desayuno (Bakery + Beverages + Eggs/Meat), Combo Limpieza (Cleaning + Beauty & Hygiene), Combo Familiar (Fruits + Foodgrains + Snacks + Baby Care). Los combos se ofrecen con descuento del 10-15% sobre la compra individual.

Beneficio esperado: Incrementar el número de categorías por transacción y el monto total, aprovechando patrones de compra validados por los datos.

Modelo: Recomendación Personalizada por KNN

Segmento objetivo: Todos los clientes con historial de compras.

Descripción: Usar el sistema de KNN Hamming implementado en el Ejercicio 3 para generar recomendaciones personalizadas en tiempo real. Cuando un cliente consulta su carrito o histórico, el sistema busca al vecino más similar y sugiere las categorías que el vecino compra pero el cliente no. En nuestro ejercicio se demostró que este sistema recomienda exitosamente Eggs/Meat y Cleaning a clientes que no las habían explorado.

Beneficio esperado: Expandir el alcance de categorías por cliente, incrementar el ticket promedio, y mejorar la experiencia de compra con sugerencias relevantes.

Modelo: Clasificación Automática de Clientes

Segmento objetivo: Gestión interna del negocio.

Descripción: Usar el algoritmo KNN Euclidiana (Ejercicio 2) para clasificar automáticamente a cada nuevo cliente que registra su primera compra. Basado en su gasto y cantidad de productos, el sistema lo asigna a Gasto Alto o Gasto Bajo, activando automáticamente la estrategia de marketing correspondiente. Los clientes de Gasto Alto reciben programa VIP, los de Gasto Bajo reciben promociones de incremento.

Beneficio esperado: Segmentación instantánea sin intervención humana, estrategias diferenciadas desde el primer día, y optimización del presupuesto de marketing.

Ofertas para Potenciar Ventas y Reducir Mermas

Las siguientes ofertas están diseñadas usando los itemsets frecuentes y reglas de asociación encontrados. El objetivo es impulsar las categorías con menor rotación combinándolas con las de mayor popularidad.

Ofertas para Reducir Mermas en Perecederos

Combo	Categorías incluidas	Descuento	Justificación
Combo Desayuno	Bakery + Beverages + Eggs/Meat	15%	Itemset 3 de alto soporte
Combo Despensa	Foodgrains + Fruits/Veg + Snacks	12%	Categorías base de alta rotación
Combo Bebé	Baby Care + Bakery + Cleaning	20%	Baby Care tiene menor soporte
Combo Gourmet	Gourmet + Beverages + Snacks	18%	Gourmet tiene bajo soporte

Ofertas de Cross-Selling (Alto Soporte)

Regla base	Oferta	Descuento	Confianza
Bakery → Beverages	Lleva Bakery, 10% en Beverages	10%	~83%
Fruits → Foodgrains	Lleva Frutas, 8% en Granos	8%	~86%
Cleaning → Beauty	Lleva Limpieza, 12% en Higiene	12%	~67%
Snacks → Beverages	Lleva Snacks, 10% en Bebidas	10%	~83%

Ofertas por Tipo de Cliente

Segmento	Oferta	Mecánica	Objetivo
Gasto Alto	Acceso anticipado a productos Gourmet nuevos	Exclusividad	Retención
Gasto Alto	Envío gratis en compras de 5+ categorías	Incentivo	Diversificación
Gasto Bajo	Cupón 20% en categoría que aún no compra (KNN Hamming)	Cupón	Expansión
Gasto Bajo	Combo de bienvenida con 3 categorías populares	Descuento 15%	Incremento

CASOS DE USO

Los casos de uso representan las funcionalidades principales del sistema, describiendo cómo un actor (analista de datos o gerente de supermercado) interactúa con la base de datos y los algoritmos implementados para resolver problemas reales del negocio. Se definieron 3 casos de uso principales, cada uno correspondiente a un ejercicio implementado en Python.

Diagrama General de Casos de Uso

Actor principal: Analista de Datos / Gerente de Supermercado

Sistema: Base de datos PostgreSQL + Scripts de Python + Dashboard Web

CU	Nombre	Descripción breve
CU-01	Reglas de Asociación	Descubrir qué categorías de productos se compran juntas calculando soporte, confianza y lift sobre 400,000 ventas
CU-02	Clasificación de Clientes	Clasificar un nuevo cliente como Gasto Alto o Gasto Bajo usando KNN con K=3 y distancia Euclidiana
CU-03	Recomendación de Categorías	Recomendar categorías de productos que el cliente aún no ha comprado usando KNN con K=1 y distancia de Hamming

CU-01: Reglas de Asociación entre Categorías

Ficha del Caso de Uso

Campo	Descripción
Identificador	CU-01
Nombre	Reglas de Asociación entre Categorías de Productos
Actor principal	Analista de Datos / Gerente de Supermercado
Precondiciones	Las tablas articulos, ventas y detalle_ventas deben estar cargadas en PostgreSQL con datos válidos. Se requieren al menos las 400,000 ventas generadas.
Postcondiciones	Se genera un archivo CSV con las 90 reglas de asociación y 3 gráficas PNG (barras de lift, heatmap de confianza, scatter soporte vs confianza).
Descripción	El analista ejecuta el script ejercicio1_reglas_asociacion.py para descubrir qué categorías de productos se compran juntas con mayor

	frecuencia. El sistema consulta la BD, calcula las tres métricas fundamentales (soporte, confianza, lift) para todos los pares posibles de categorías, y presenta los resultados ordenados por relevancia.
Script	ejercicio1_reglas_asociacion.py
Algoritmo	Apriori (implementado con itertools.combinations + evaluación de soporte)

Problema que Resuelve

En un supermercado real, el gerente necesita tomar decisiones sobre dónde colocar los productos en los estantes, qué promociones cruzadas ofrecer y qué combos armar. Sin datos, estas decisiones se basan en intuición: «me parece que la gente que lleva pan también lleva leche». Este caso de uso reemplaza la intuición con datos: analiza las 400,000 ventas y calcula matemáticamente la fuerza de asociación entre cada par de categorías.

Físicamente en la tienda, un gerente solo puede observar unas pocas compras al día. Con la base de datos, se analizan todas las 400,000 ventas en minutos y se obtienen resultados estadísticamente significativos.

Flujo de Ejecución Detallado

1. El script se conecta a PostgreSQL usando SQLAlchemy con la cadena de conexión configurada.
2. Ejecuta la consulta SQL: `SELECT d.id_venta, a.category FROM detalle_ventas d JOIN articulos a ON d.id_articulo = a.id_articulo`. Esto devuelve todas las combinaciones de venta-categoría (millones de filas).
3. Agrupa por `id_venta` y genera un frozenset de categorías únicas para cada venta. Resultado: 400,000 conjuntos de categorías.
4. Calcula el soporte individual de cada categoría: cuenta en cuántas ventas aparece cada una de las 10 categorías y divide entre el total (400,000).
5. Para cada par de categorías (45 pares posibles con 10 categorías), cuenta cuántas ventas contienen AMBAS categorías simultáneamente. Calcula el soporte conjunto.
6. Para cada par genera 2 reglas ($X \rightarrow Y$ y $Y \rightarrow X$), calculando: $\text{Confianza} = \text{Soporte}(X \cup Y) / \text{Soporte}(X)$ y $\text{Lift} = \text{Confianza} / \text{Soporte}(Y)$. Total: 90 reglas.
7. Ordena las 90 reglas por lift descendente e imprime la tabla de las top 20.
8. Muestra un ejemplo detallado paso a paso con la regla de mayor lift, mostrando los cálculos intermedios de forma idéntica al formato de libreta.
9. Genera 3 gráficas con Matplotlib: barras horizontales de las top 20 reglas por lift (con línea roja en $\text{lift}=1$), heatmap de confianza 10×10 (mapa de calor donde cada celda es la confianza de fila $\rightarrow$ columna), y scatter plot de soporte vs confianza (donde el tamaño y color de cada punto representan el lift).
10. Guarda los resultados en `ejercicio1_reglas_asociacion.csv` y las 3 imágenes PNG.

Métricas Calculadas

Soporte

Fórmula: $\text{Soporte}(X \cup Y) = \text{Número de ventas que contienen } X \text{ y } Y / \text{Total de ventas (400,000)}$. El soporte mide qué tan común es una combinación. Un soporte de 0.85 significa que el 85% de todas las ventas incluyen ambas categorías. Soportes muy bajos indican combinaciones raras; soportes altos indican combinaciones ubicuas.

Confianza

Fórmula: $\text{Confianza}(X \rightarrow Y) = \text{Soporte}(X \cup Y) / \text{Soporte}(X)$. La confianza es asimétrica: la confianza de $X \rightarrow Y$ es diferente a la de $Y \rightarrow X$. Una confianza de 0.92 en la regla Bakery $\rightarrow$ Fruits/Veg significa que el 92% de las ventas que incluyen Bakery también incluyen Fruits/Veg. Es una probabilidad condicional: $P(Y|X)$.

Lift

Fórmula: $\text{Lift}(X \rightarrow Y) = \text{Confianza}(X \rightarrow Y) / \text{Soporte}(Y)$. El lift elimina el efecto de la popularidad individual de Y. Si Y aparece en el 90% de las ventas, una confianza del 90% para $X \rightarrow Y$ no es impresionante porque Y ya aparece casi siempre. El lift corrige esto: si $\text{lift} > 1$, la asociación es más fuerte que el azar; si $\text{lift} = 1$, X e Y son independientes; si $\text{lift} < 1$, la presencia de X reduce la probabilidad de Y.

Resultados Obtenidos

La regla con mayor lift fue Baby Care $\rightarrow$ Gourmet (~1.14), lo que indica que cuando productos de Baby Care están presentes en una venta, la probabilidad de que también haya productos Gourmet supera en un 14% lo esperado por azar. Esto tiene sentido en un contexto real: las familias con bebés tienden a ser familias jóvenes que también exploran productos gourmet y especializados.

Las reglas con mayor confianza involucran Fruits & Vegetables como consecuente (confianza > 90%), lo cual refleja que esta categoría tiene el soporte individual más alto y aparece en la gran mayoría de ventas. Esto demuestra la importancia del lift: sin él, «todo $\rightarrow$ Fruits/Veg» parecería la regla más importante cuando en realidad es simplemente la más obvia.

Gráficas Generadas

- ejercicio1_reglas_lift.png: Barras horizontales de las top 20 reglas ordenadas por lift, con la confianza anotada a la derecha de cada barra y una línea punteada roja en lift=1 como referencia de independencia.
- ejercicio1_heatmap_confianza.png: Mapa de calor 10×10 donde el eje vertical es el antecedente (X), el eje horizontal es el consecuente (Y), y cada celda muestra el valor numérico de la confianza con colores que van de amarillo claro (baja) a rojo oscuro (alta).
- ejercicio1_scatter_soporte_confianza.png: Gráfica de dispersión donde el eje X es el soporte conjunto, el eje Y es la confianza, el tamaño de cada punto es proporcional al lift, y el color indica la magnitud del lift (azul=bajo, rojo=alto). Las reglas más interesantes están en la esquina inferior izquierda (bajo soporte pero alto lift).

CU-02: Clasificación de Clientes con KNN Euclidiana

Ficha del Caso de Uso

Campo	Descripción
Identificador	CU-02
Nombre	Clasificación de Clientes por Nivel de Gasto usando KNN con Distancia Euclidiana
Actor principal	Gerente de Supermercado / Departamento de Marketing
Precondiciones	La tabla ventas debe contener las 400,000 ventas con id_cliente, total_venta y cantidad_productos. Se requieren clientes con al menos 3 compras para datos significativos.
Postcondiciones	El nuevo cliente queda clasificado como 'Gasto Alto' o 'Gasto Bajo'. Se generan 2 gráficas PNG.
Descripción	El gerente necesita clasificar a un nuevo cliente para asignarle el programa de fidelización correspondiente. El sistema consulta los datos históricos de 200 clientes, calcula la distancia Euclidiana del nuevo cliente a cada uno, selecciona los K=3 más cercanos y clasifica por votación mayoritaria.
Script	ejercicio2_knn_euclidiana.py
Algoritmo	K-Nearest Neighbors con K=3 y Distancia Euclidiana

Problema que Resuelve

Un supermercado con 50,000 clientes no puede asignar manualmente un nivel de servicio a cada uno. Un cajero no puede recordar quién gasta mucho y quién poco. Sin embargo, para estrategias de marketing efectivas es fundamental distinguir entre clientes de alto valor (que merecen programa VIP, descuentos exclusivos, atención preferencial) y clientes de bajo valor (que necesitan incentivos para incrementar su gasto).

Este caso de uso automatiza esa clasificación: cuando un nuevo cliente realiza sus primeras compras, el sistema calcula automáticamente qué tipo de cliente es basándose en cuánto gasta y cuántos productos lleva, comparándolo con clientes existentes cuyo comportamiento ya se conoce.

Concepto: Distancia Euclidiana

La distancia Euclidiana es la distancia «en línea recta» entre dos puntos en un espacio de N dimensiones. Es la generalización del teorema de Pitágoras. Para nuestro caso con 2 dimensiones (gasto y productos):

$$d = \sqrt{(\text{gasto}_1 - \text{gasto}_2)^2 + (\text{productos}_1 - \text{productos}_2)^2}$$

Ejemplo: si el cliente nuevo tiene gasto=\$4,450 y productos=16.1, y el cliente C5 tiene gasto=\$4,429 y productos=15.6, la distancia es: $d = \sqrt{(4450-4429)^2 + (16.1-15.6)^2} = \sqrt{441 + 0.25} = \sqrt{441.25} \approx 21$.

Este valor se compara con las distancias a todos los demás clientes para encontrar los 3 más cercanos.

Flujo de Ejecución Detallado

11. El script se conecta a PostgreSQL y ejecuta una consulta que agrupa ventas por id_cliente, calculando AVG(total_venta) como gasto_promedio y AVG(cantidad_productos) como productos_promedio. Solo incluye clientes con al menos 3 compras (HAVING COUNT(*) >= 3). Toma una muestra de 200 clientes.
12. Calcula la mediana del gasto_promedio de los 200 clientes. Los que están por encima se etiquetan como 'Gasto Alto', los de abajo como 'Gasto Bajo'.
13. Selecciona 6 clientes como muestra de ejemplo (para la demostración paso a paso) y define un nuevo cliente a clasificar con el gasto y productos promedio de la muestra.
14. Para cada uno de los 6 clientes de ejemplo, calcula la distancia Euclidiana mostrando la fórmula completa: la resta, el cuadrado, la suma y la raíz cuadrada. Esto replica el formato del ejercicio de la libreta.
15. Ordena los 6 clientes por distancia ascendente y selecciona los K=3 más cercanos.
16. Realiza la votación: cuenta cuántos de los 3 vecinos son 'Gasto Alto' y cuántos son 'Gasto Bajo'. La clase con más votos gana.
17. Repite el proceso con los 200 clientes completos para generar la segunda gráfica con el círculo de radio K.
18. Genera 2 gráficas PNG con Matplotlib.

Resultados Obtenidos

En el ejemplo ejecutado, el nuevo cliente fue clasificado como 'Gasto Bajo'. Los 3 vecinos más cercanos tenían distancias de 21, 116 y 169 unidades, y todos pertenecían a la clase 'Gasto Bajo', dando una votación unánime de 3-0. La mediana de gasto se ubicó alrededor de \$4,400, lo cual divide a los 200 clientes en dos grupos equilibrados.

La gráfica de 200 clientes muestra claramente la separación entre los dos grupos: los clientes de Gasto Alto (rojo) se concentran a la derecha del gráfico con gastos superiores a la mediana, mientras que los de Gasto Bajo (azul) se concentran a la izquierda. El círculo verde alrededor del nuevo cliente visualiza la 'zona de vecindad' del algoritmo.

Gráficas Generadas

- ejercicio2_knn_euclidiana.png: Scatter plot con 6 clientes de ejemplo. Puntos rojos = Gasto Alto, cuadrados azules = Gasto Bajo, estrella verde = nuevo cliente. Líneas punteadas conectan al nuevo cliente con sus 3 vecinos más cercanos, con la distancia anotada sobre cada línea. Cada punto tiene una etiqueta (C1 a C6).
- ejercicio2_knn_200_clientes.png: Scatter plot con los 200 clientes reales de la BD. Círculos pequeños rojos = Gasto Alto, cuadrados pequeños azules = Gasto Bajo, estrella verde = nuevo cliente. Un círculo punteado verde con radio igual a la distancia del 3er vecino delimita la zona de decisión del algoritmo.

CU-03: Recomendación de Categorías con KNN Hamming

Ficha del Caso de Uso

Campo	Descripción
Identificador	CU-03
Nombre	Recomendación de Categorías de Productos mediante KNN con Distancia de Hamming
Actor principal	Sistema de Recomendación Automático / Cliente del Supermercado
Precondiciones	Las tablas ventas, detalle_ventas y articulos deben contener datos válidos. Se requieren ventas con variedad de categorías (5-9 por venta) para que las recomendaciones sean significativas.
Postcondiciones	Se identifican categorías de productos que el cliente aún no ha comprado pero que clientes similares sí compran. Se generan 3 gráficas PNG.
Descripción	El sistema toma la última compra de un cliente (Venta X), la representa como un vector binario de 10 dimensiones (una por categoría), busca la venta más similar usando distancia de Hamming, y recomienda las categorías que el vecino tiene pero la Venta X no.
Script	ejercicio3_knn_hamming.py
Algoritmo	K-Nearest Neighbors con K=1 y Distancia de Hamming

Problema que Resuelve

Los supermercados quieren que cada cliente explore todas las categorías de productos disponibles, no solo las que ya conoce. Si un cliente siempre compra en 5 categorías pero nunca en las otras 5, está dejando de gastar potencialmente el doble. El problema es que no se puede recomendar lo mismo a todos: cada cliente tiene un perfil diferente y las recomendaciones deben ser personalizadas.

Este caso de uso funciona como los sistemas de recomendación de Netflix o Spotify: encuentra al «usuario más parecido a ti» y te sugiere lo que ese usuario disfruta pero tú aún no has probado. En nuestro caso, si tu vecino más similar compra Eggs/Meat y Cleaning pero tú no, el sistema te recomienda explorar esas categorías.

Físicamente en una tienda, un empleado no puede saber qué compra cada cliente ni compararlos entre sí. Con la base de datos y el algoritmo, esta comparación se hace automáticamente entre 400,000 ventas en segundos.

Concepto: Distancia de Hamming

La distancia de Hamming entre dos vectores binarios (secuencias de 0 y 1) es simplemente el número de posiciones donde los valores son diferentes. Es la métrica ideal para datos categóricos binarios porque no asume relación de magnitud entre las dimensiones (no tiene sentido decir que 'Bakery está más lejos de Baby que de Beverages').

Ejemplo: Venta X = (0,1,0,1,0,0,1,1,0,1) y Venta A = (0,1,0,1,1,1,1,1,0,1). Difieren en las posiciones 5 y 6 (Cleaning y Eggs/Meat), donde X tiene 0 y A tiene 1. Distancia de Hamming = 2.

Concepto: Vector Binario de Categorías

Cada venta se representa como un vector de 10 posiciones, una por cada categoría de productos, ordenadas alfabéticamente: [Baby, Bakery, Beauty, Beverages, Cleaning, Eggs/Meat, Foodgrains, Fruits/Veg, Gourmet, Snacks]. Si la venta incluye productos de esa categoría, la posición vale 1; si no, vale 0.

Este método permite representar información categórica compleja (qué categorías compró un cliente) como un vector numérico simple sobre el cual se pueden calcular distancias.

Flujo de Ejecución Detallado

19. El script consulta detalle_ventas JOIN articulos para obtener las categorías presentes en cada venta. Para cada id_venta se genera un set de categorías únicas y se cuenta cuántas tiene.
20. Selecciona la Venta X: una venta con 5-6 categorías (no todas las 10), para que haya espacio para recomendar las faltantes. La convierte a vector binario.
21. Selecciona 4 ventas vecinas (A, B, C, D) que tengan 7-9 categorías y que tengan al menos 1 categoría que X NO tiene. Esto garantiza que la recomendación funcionará.
22. Para cada vecino, calcula la distancia de Hamming contando las posiciones donde difieren. Muestra el cálculo paso a paso: qué posiciones son diferentes, qué categorías representan, y cuál es la distancia total.
23. Ordena los vecinos por distancia ascendente. Con K=1, selecciona el más cercano.
24. Compara los vectores de X y el vecino ganador. Identifica las posiciones donde X tiene 0 y el vecino tiene 1: esas son las categorías que el vecino compra pero X no.
25. Genera la recomendación: «Se recomienda al cliente explorar las categorías Eggs/Meat y Cleaning».
26. Genera 3 gráficas PNG con Matplotlib.

Resultados Obtenidos

En la ejecución del ejercicio, la Venta X tenía 5 categorías: Bakery, Beverages, Foodgrains, Fruits/Veg y Snacks. Le faltaban 5 categorías: Baby, Beauty, Cleaning, Eggs/Meat y Gourmet.

El vecino más cercano fue la Venta A con distancia de Hamming = 2. La Venta A tenía 7 categorías, incluyendo Cleaning y Eggs/Meat que la Venta X no tenía. Las diferencias estaban exactamente en estas dos posiciones del vector.

El sistema recomendó exitosamente las categorías Eggs/Meat y Cleaning, que son categorías complementarias lógicas: quien compra alimentos básicos (Foodgrains, Fruits/Veg, Bakery) probablemente también necesita proteína (Eggs/Meat) y productos de limpieza (Cleaning).

Gráficas Generadas

- ejercicio3_vectores_binarios.png: Heatmap de 5 filas (Venta X y 4 vecinos) por 10 columnas (categorías). Verde oscuro = 1 (compró), claro = 0 (no compró). Las posiciones donde X y el vecino ganador (A) difieren están resaltadas con bordes rojos gruesos. Permite ver de un vistazo dónde están las diferencias.
- ejercicio3_distancias_hamming.png: Barras verticales mostrando la distancia de Hamming de cada vecino a X. El vecino más cercano aparece en verde, los demás en azul. El número de distancia aparece sobre cada barra. Permite comparar rápidamente qué vecino es más similar.
- ejercicio3_recomendacion.png: Tres paneles horizontales lado a lado. Panel izquierdo: lo que tiene Venta X (barras verdes = tiene, rosa = no tiene). Panel central: lo que tiene el vecino A (barras azules = tiene, rosa = no tiene). Panel derecho: Recomendación (barras rojas con «★ Recomendar» en las categorías Eggs/Meat y Cleaning). Visualización clara del resultado final.

Definición del Sistema: Dashboard Web

Propósito

Se desarrolló una página web como interfaz de presentación para los 3 casos de uso. El dashboard permite navegar entre los ejercicios mediante pestañas, mostrando las gráficas generadas por los scripts de Python junto con tarjetas informativas que explican los conceptos y fórmulas de cada algoritmo.

Especificaciones Técnicas

Propiedad	Valor
Archivo	dashboard_casos_de_uso.html (archivo único autocontenido)
Tamaño	~1.2 MB (por las 8 imágenes embebidas en Base64)
Tecnologías	HTML5, CSS3 (variables, Grid, Flexbox), JavaScript (vanilla)
Fuentes	DM Sans (texto) y JetBrains Mono (fórmulas), vía Google Fonts
Tema visual	Dark theme con colores morado (CU-01), cyan (CU-02), rosa (CU-03)
Imágenes	8 archivos PNG embebidos en Base64 (no requiere archivos externos)
Compatibilidad	Chrome, Firefox, Safari, Edge (cualquier versión moderna)
Dependencias	Ninguna. Se abre con doble clic en cualquier computadora.

Estructura del Dashboard

Header: Título del proyecto y descripción, con fondo de gradiente sutil que combina los 3 colores de acento.

Barra de pestañas: 3 pestañas horizontales (Reglas de Asociación, KNN Euclidiana, KNN Hamming) con badges de color que identifican cada ejercicio. La pestaña activa tiene borde inferior cyan y fondo sutil.

Tarjetas informativas: Cada pestaña comienza con 3 tarjetas (info-box) que resumen los conceptos clave y fórmulas del algoritmo correspondiente. Las fórmulas usan fuente monoespaciada sobre fondo cyan sutil.

Tarjetas de imagen: Cada gráfica se presenta en una tarjeta con fondo blanco (para contrastar con el tema oscuro), esquinas redondeadas, y un caption explicativo debajo.

Animación: Al cambiar de pestaña, el contenido nuevo aparece con un fade-in de 0.3 segundos que combina opacidad y movimiento vertical.

Conclusiones

Sobre la Base de Datos y el Proceso ETL

PostgreSQL demostró ser un SGBD robusto capaz de manejar aproximadamente 7.6 millones de registros distribuidos en 4 tablas con integridad referencial completa. La estrategia de carga masiva con `pandas.to_sql()` usando parámetros `chunksize=10000` y `method='multi'` permitió insertar los ~6.8 millones de registros de `detalle_ventas` en minutos, lo cual habría tomado horas con inserciones individuales.

El proceso ETL de 5 pasos demostró que la limpieza de datos es el paso más crítico de cualquier proyecto de análisis. De los 27,555 productos originales del dataset BigBasket, solo 9,996 (36%) eran realmente de canasta básica. Sin este filtrado, los patrones encontrados por los algoritmos incluirían asociaciones espurias entre cosméticos, artículos de mascotas y productos de papelería que no representan el comportamiento real de compra en un supermercado.

La experiencia con el dataset de PROFECO (descartado por segmentación geográfica y exceso de artículos irrelevantes) confirmó que la selección adecuada del dataset es tan importante como el análisis mismo. Un dataset inadecuado produce resultados sin valor, sin importar qué tan sofisticado sea el algoritmo aplicado.

Sobre los Itemsets Frecuentes

El cálculo de 637 itemsets de nivel 1 a 5 demostró la viabilidad de aplicar Market Basket Analysis sobre un volumen significativo de datos. La estrategia de diversidad forzada (mínimo 5 categorías por ticket) fue esencial para que los itemsets de nivel 4 y 5 tuvieran soporte significativo. Sin esta restricción, los quintetos habrían tenido soporte cercano a cero y serían estadísticamente irrelevantes.

Se observó que el soporte máximo disminuye progresivamente con el nivel: ~92% para individuales, ~85% para pares, ~78% para tríos, ~65% para cuartetos y ~50% para quintetos. Esta degradación es esperada y natural, ya que es progresivamente menos probable que N categorías específicas coincidan todas en la misma venta.

Sobre las Reglas de Asociación

Las 90 reglas de asociación demostraron la importancia de usar las tres métricas de forma complementaria, no aislada. El soporte mide popularidad (qué tan común es la combinación), la confianza mide probabilidad condicional (qué tan probable es Y dado X), y el lift mide si la asociación es genuina o simplemente refleja la popularidad individual de los ítems.

Un hallazgo importante fue que todas las reglas tienen lift cercano a 1 (rango 0.98 a 1.14). Esto indica que las categorías son relativamente independientes entre sí en nuestros datos sintéticos, lo cual es consistente con la generación aleatoria ponderada por categoría. En un dataset de transacciones reales, se esperarían lifts más extremos que reflejen asociaciones genuinas del comportamiento del consumidor.

La línea roja en $\text{lift}=1$ de la gráfica de barras resultó ser una herramienta visual poderosa para distinguir asociaciones positivas ($\text{lift} > 1$, las categorías se refuerzan) de negativas ($\text{lift} < 1$, las categorías se inhiben).

Sobre KNN y sus Dos Variantes

KNN demostró ser un algoritmo versátil que puede aplicarse a problemas fundamentalmente diferentes cambiando únicamente la métrica de distancia:

Con distancia Euclidiana (Ejercicio 2): Funciona como clasificador de clientes. La distancia se calcula sobre variables numéricas continuas (gasto y productos) y la predicción se basa en votación mayoritaria de los K vecinos. Es intuitivo y fácil de explicar a stakeholders no técnicos.

Con distancia de Hamming (Ejercicio 3): Funciona como sistema de recomendación. La distancia se calcula sobre vectores binarios (compra/no compra) y la predicción se basa en las diferencias entre el cliente y su vecino más similar. Es el mismo principio que usan Netflix y Spotify para recomendar contenido.

La elección de la distancia depende del tipo de datos: Euclidiana para variables numéricas continuas (precios, cantidades, montos), Hamming para variables categóricas binarias (compra/no compra, sí/no), y existen otras como Manhattan, Coseno o Minkowski para casos específicos.

Sobre la Visualización y Presentación

Las 14 gráficas generadas con Matplotlib y el dashboard web con pestañas demostraron que la visualización no es un complemento opcional sino una parte integral del análisis. Un heatmap de confianza 10×10 comunica en segundos lo que una tabla de 90 filas tardaría minutos en transmitir. El panel de recomendación del Ejercicio 3, con sus tres secciones lado a lado (tiene/tiene/recomienda), hace inmediatamente comprensible un concepto que en texto resultaría abstracto.

La decisión de crear un archivo HTML autocontenido (con imágenes embebidas en Base64) resultó práctica para la portabilidad: el dashboard se puede enviar por correo, copiar a USB o abrir en cualquier computadora sin dependencias externas.

Aprendizaje Principal

El proyecto demostró que el valor de los datos no está en su volumen sino en el proceso completo de obtenerlos, limpiarlos, almacenarlos, analizarlos, visualizarlos e interpretarlos. Los 27,555 productos del CSV original no dicen nada por sí solos. Fue necesario el proceso ETL para convertirlos en 9,996 productos útiles, la generación sintética para crear 400,000 transacciones realistas, los algoritmos para descubrir patrones ocultos, y las visualizaciones para comunicar esos patrones de forma comprensible.

Este flujo completo — desde el dato crudo hasta la decisión de negocio — es exactamente lo que una organización necesita implementar para convertirse en una empresa data-driven. Y las herramientas utilizadas (Python, Pandas, PostgreSQL, Matplotlib) están disponibles de forma

gratuita, lo que demuestra que el análisis avanzado de datos no requiere inversiones millonarias sino conocimiento y metodología.

Referencias

Agrawal, R., Imielinski, T., & Swami, A. (1993). Mining Association Rules Between Sets of Items in Large Databases. Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data, 207-216.

Agrawal, R., & Srikant, R. (1994). Fast Algorithms for Mining Association Rules. Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), 487-499.

BigBasket Product List Dataset. (2023). Kaggle. <https://www.kaggle.com/datasets/surajjha101/bigbasket-entire-product-list-28k-datapoints>

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21-27.

Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Gravitar. (2025). Market Basket Analysis: Cómo los supermercados utilizan los datos de compra. Recuperado de <https://gravitar.biz/>

Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann.

Heaton, J. (2017). Comparing Dataset Characteristics that Favor the Apriori, Eclat or FP-Growth Frequent Itemset Mining Algorithms. IEEE SoutheastCon 2017, 1-7.

Inmon, W. H. (2005). Building the Data Warehouse (4th ed.). John Wiley & Sons.

Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd ed.). John Wiley & Sons.

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.

Pandas Development Team. (2025). pandas: powerful Python data analysis toolkit. <https://pandas.pydata.org/>

PostgreSQL Global Development Group. (2025). PostgreSQL: The World's Most Advanced Open Source Relational Database. <https://www.postgresql.org/>

Santa Cruz, J. (2013). Market Basket Analysis y Patrones de Compra del Consumidor. Análisis de Minería de Datos Aplicada al Comercio Minorista.

Scikit-learn Developers. (2025). scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>

SQLAlchemy Authors. (2025). SQLAlchemy — The Database Toolkit for Python. <https://www.sqlalchemy.org/>

Tan, P.-N., Steinbach, M., & Kumar, V. (2019). Introduction to Data Mining (2nd ed.). Pearson.

Vapnik, V. N. (1995). The Nature of Statistical Learning Theory. Springer-Verlag.

Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2017). Data Mining: Practical Machine Learning Tools and Techniques (4th ed.). Morgan Kaufmann.